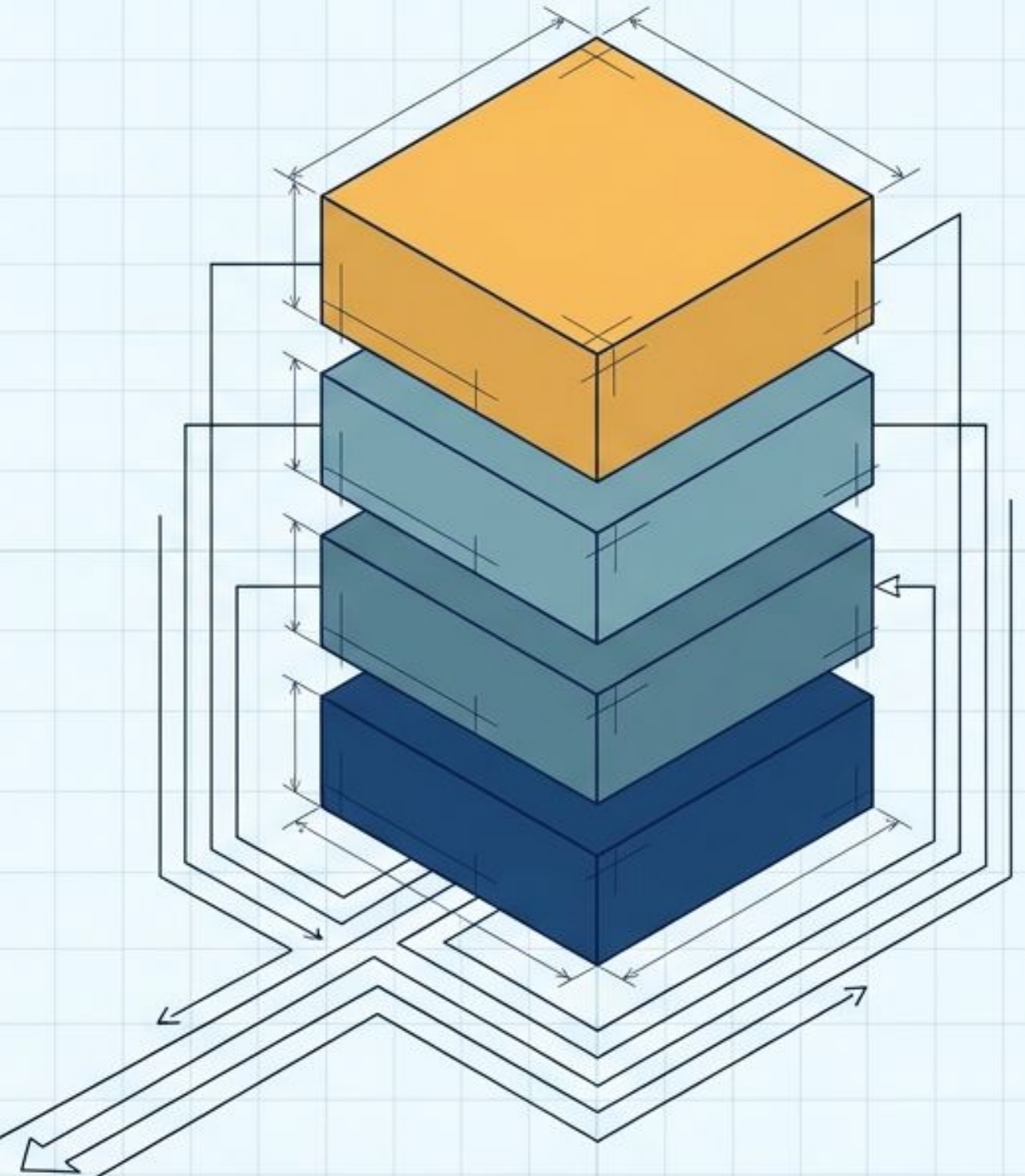


Data Engineering Blueprint: De Local a Cloud

Guía de referencia técnica y modelos mentales para el diseño, orquestación y tolerancia a fallos en ecosistemas distribuidos modernos.



Apache Airflow 3.x
Cerebro/Orquestador

**Apache
Airflow 3.x**

Orquesta Tareas
(SparkSubmitOperator)

**Apache
Spark 3.5.8**

Apache Spark 3.5.8
Motor de Cómputo local[*]

Monitorea/Registra Estado

**PostgreSQL
15**

PostgreSQL 15
Metadata & Data Marts

Lee/Escribe Datos

MinIO S3
Data Lake,
Puertos 9000/9001

Ubuntu 24.04 LTS VM (2 Cores, 4GB RAM)

Entorno base: Python 3.12.3.
Modo Standalone.

Matriz de Decisión de Arquitectura

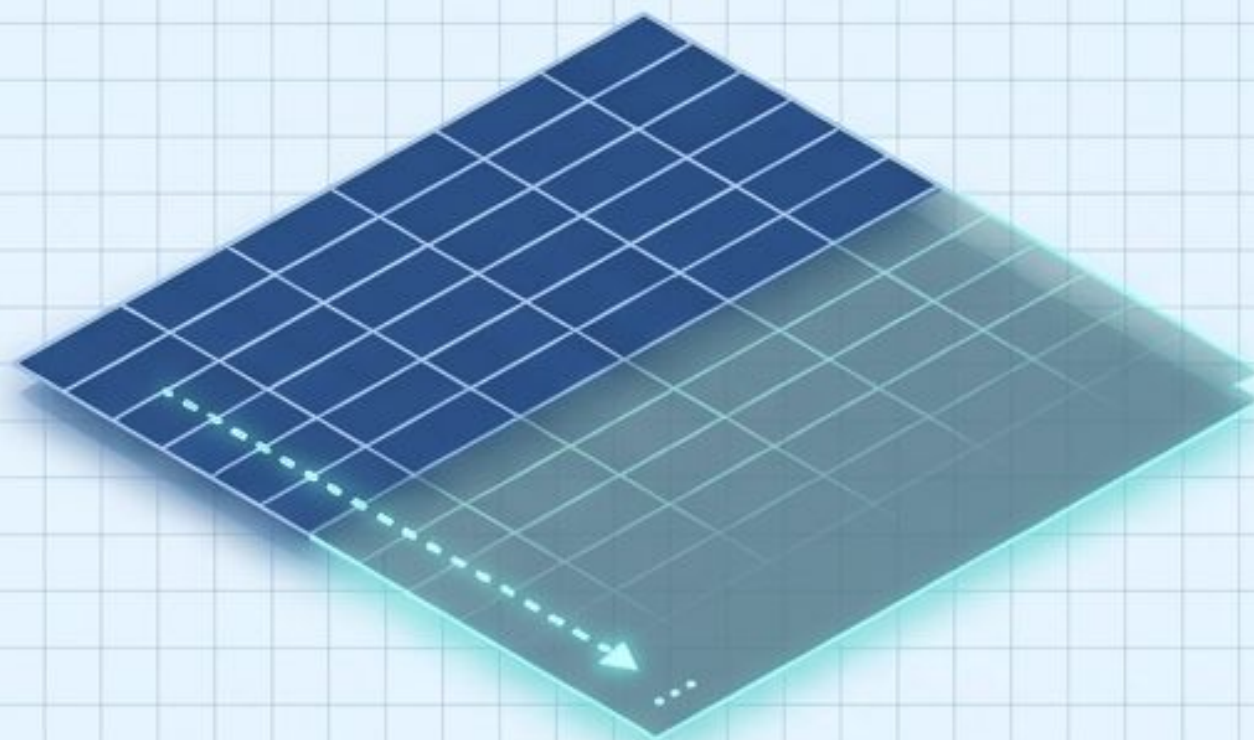


Procesamiento Batch (Lotes)	Structured Streaming	Enfoque Híbrido
100% Latencia: Alta (Horas/días)	10% Latencia: Baja (Milisegundos/minutos)	<pre>graph LR; Batch[Batch] --> Integration[Integración/Unificación]; Streaming[Streaming] --> Integration; Integration --> Resultado[Resultado Combinado];</pre>
75% Volumen: Grandes volúmenes acotados (Terabytes/Petabytes finitos).	100% ∞ Volumen: Infinito continuo	
30% Complejidad: Baja/Media	90% Complejidad: Alta (Requiere control de estado)	
20% Costo: Eficiente (Cómputo efímero bajo demanda).	95% Costo: Elevado (Infraestructura activa 24/7 o micro-batches frecuentes).	

Combina ingestión masiva con alertas en tiempo real. Máxima complejidad de sincronización.

El Motor de Cómputo: Structured Streaming

La Definición: Tabla Infinita (Unbounded)



Spark trata los datos en streaming usando la misma API de DataFrames que en batch, anexando registros constantemente y optimizando las consultas de forma incremental.

El Modelo Mental: Enciclopedia vs. Pantalla Digital



Enciclopedia
(Batch)

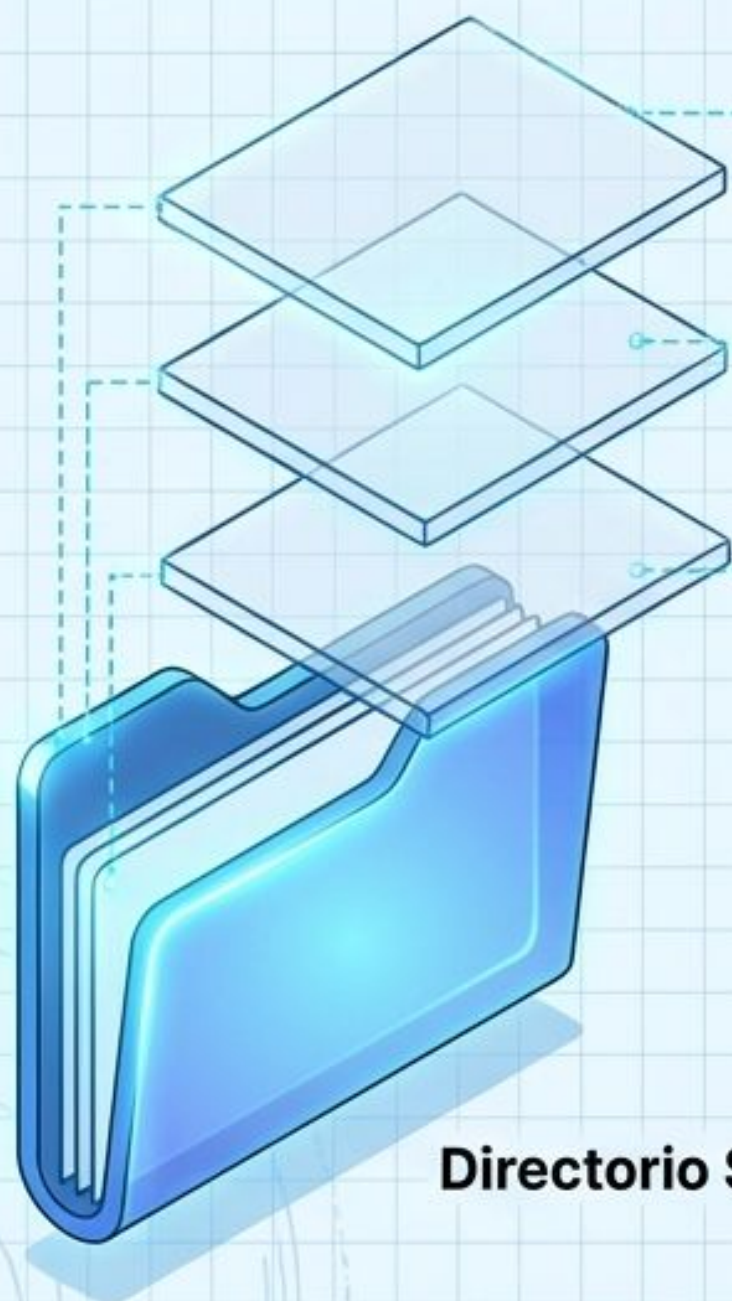


Pantalla Digital
(Streaming)



Antipatrón Crítico: Intentar realizar ordenamientos globales (sorts) o múltiples agregaciones encadenadas sin definir ventanas temporales (watermarks).

Tolerancia a Fallos: Anatomía del Checkpoint



/offsets

Qué datos ya se leyeron del origen.

/commits

Qué micro-batches ya finalizaron.

/state

El estado actual de la agregación.

Directorio S3/MinIO



El “Save Game” del Sistema: Si la máquina virtual se apaga repentinamente por un corte de energía, Spark no inicia la computación desde cero. Recupera el último nivel guardado con todo el inventario intacto.



Corrupción de Estado: Modificar la estructura interna del código (DAG lógico) y reiniciar usando un directorio de checkpoint viejo romperá la compatibilidad estructural.

Triggers: Balanceando Latencia y Costo

Light Studio Blueprint

`processingTime='10 seconds'`



Un autobús que sale exactamente cada 10 minutos, vaya vacío o lleno.

`availableNow=True`



Recoge la fila exacta de ese instante, hace un viaje único y apaga su motor hasta el día siguiente.

Continuous Processing

Latencia sub-milisegundo real continua.



Cuello de Botella: Fijar un '`processingTime=1 second`' en hardware limitado congelará la cola. Spark tardará más en escribir metadatos en Delta que el tiempo del trigger. Usar '`maxFilesPerTrigger`'.

Apache Airflow: El Cerebro Orquestador

El Chef Ejecutivo: Posee la receta maestra (DAG), dicta los pasos de forma ordenada a los cocineros de línea (Operators) y supervisa que no se sirva el plato hasta que todos los componentes estén perfectamente cocinados.



Voz:
XComs (Metadatos JSON)

Cerebro:
API Server &
Scheduler

Memoria:
Metadata DB
(Postgres)

Músculo:
Workers
(Operators:
Bash, Python)

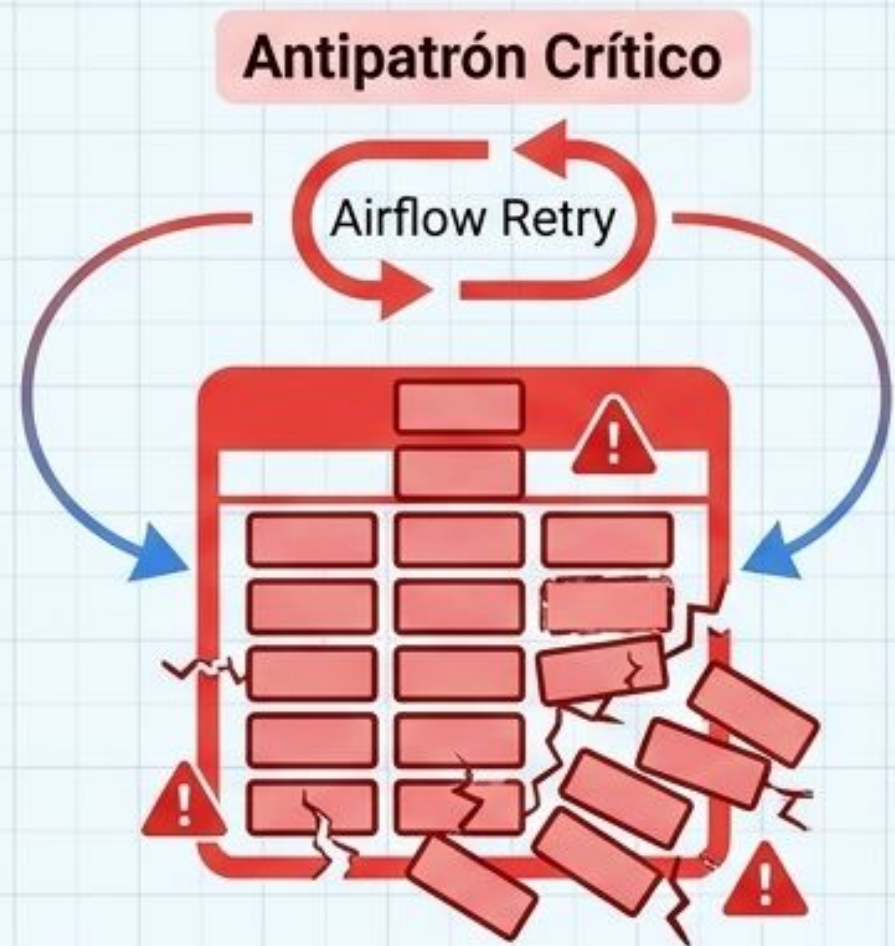
Sentidos:
Sensors
(Espera activa)



Scheduler Congelado:
Escribir consultas SQL masivas o peticiones HTTP de bloqueo directo en el cuerpo principal del script Python del DAG (fuera de las funciones de tareas u operadores). Esto degrada drásticamente el rendimiento del Scheduler, congelando la UI.

El Principio de Idempotencia

Garantiza que un pipeline produzca exactamente el mismo resultado final sin duplicar registros, sin importar cuántas veces se reejecute ante fallos.



```
df.write.mode('append').save()
```

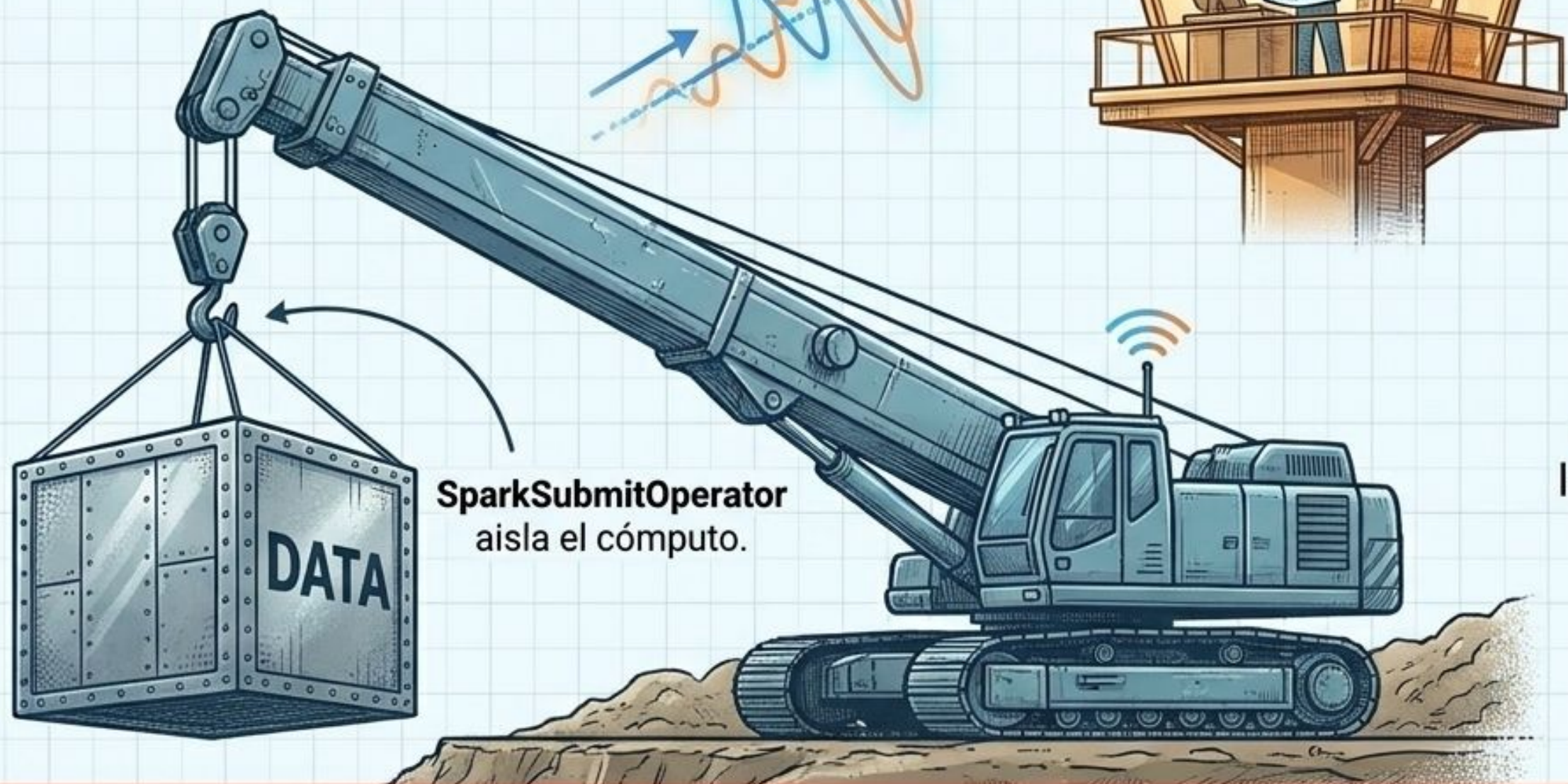
Sin transacciones ACID, los reintentos por fallos de red generan registros duplicados irreversibles.



```
.mode('overwrite')  
MERGE INTO (Delta Lake)
```

Sobreescritura completa de particiones o uso de transacciones ACID para actualizar coincidencias e insertar nuevos de forma limpia.

Integración Spark + Airflow: El Director y la Grúa



Torre de Control (Airflow): Emite la orden de trabajo parametrizada y vigila el log. No procesa la data.

La Grúa (Spark): Ejecuta la carga pesada (memoria, cores) aislada en su entorno nativo.



Falsos Positivos (SUCCESS): Ocultar errores de Spark mediante bloques try-except vacíos en Python. Airflow marcará éxito pero no habrá datos. Siempre propagar la excepción o usar `sys.exit(1)`.

Equivalencias en la Nube: Escalando el Blueprint

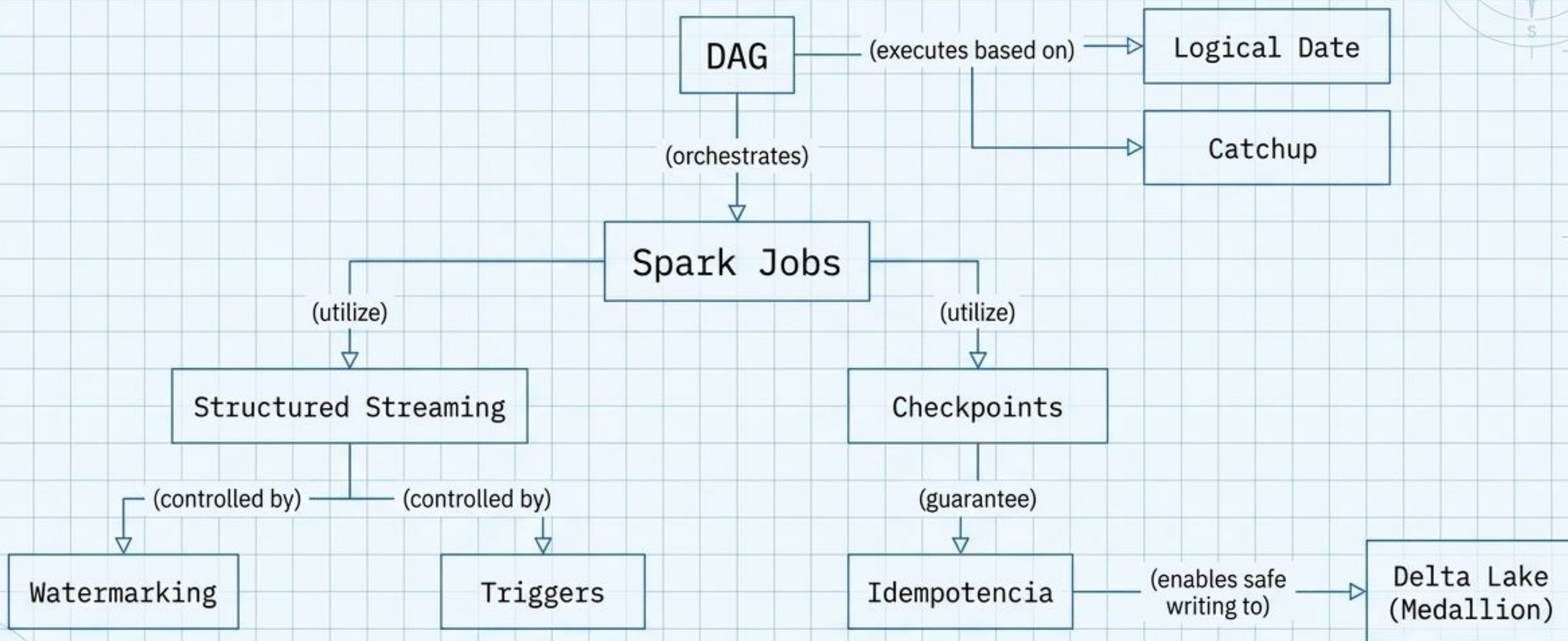
Pasar de hornear pan artesanal (VM Local) a contratar una panadería industrial global bajo demanda (Cloud Serverless).

Entorno Local	AWS	GCP	Azure / Unificado
VM (Ubuntu)	EC2	Compute Engine	Azure VMs
Airflow Local	MWAA	Cloud Composer	Astronomer / ADF
Spark Local	EMR	Dataproc	Databricks (Líder)
MinIO Local	S3	GCS	ADLS Gen2



Facturación Catastrófica: Dejar clústeres masivos encendidos 24/7 para un pipeline de 20 minutos. Exigir políticas de auto-termination o clústeres efímeros.

Síntesis del Ecosistema Topológico



Un sistema de datos robusto no es solo código; es la perfecta coreografía entre almacenamiento, orquestación, cómputo y tolerancia a fallos.